

AyED 2025

ACTIVIDAD COMPLEMENTARIA

PIENSA COMO UN PROGRAMADOR

Resolviendo el acertijo del Zorro, la Gallina y el Maíz



Un granjero con un zorro, un ganso y un saco de maíz necesita cruzar un río. El granjero tiene un bote de remos, pero solo hay espacio para él y uno de sus tres objetos. Desafortunadamente, tanto el zorro como el ganso tienen hambre. El zorro no puede quedarse solo con el ganso, o este se lo comerá. Del mismo modo, el ganso no puede quedarse solo con el saco de maíz, o se lo comerá. ¿Cómo logra el granjero cruzar el río con todo?

¿Puedes determinar si hay restricciones en el problema presentado ?

1- RESOLVERLO EN FORMA INDIVIDUAL. TIEMPO: 10 MIN

2- RESOLVEMOS DE FORMA COLABORATIVA CON UN PAR: 5 MIN

3- COMPARTIMOS ALGUNAS SOLUCIONES

¿QUÉ ESTRATEGIA USASTE ?

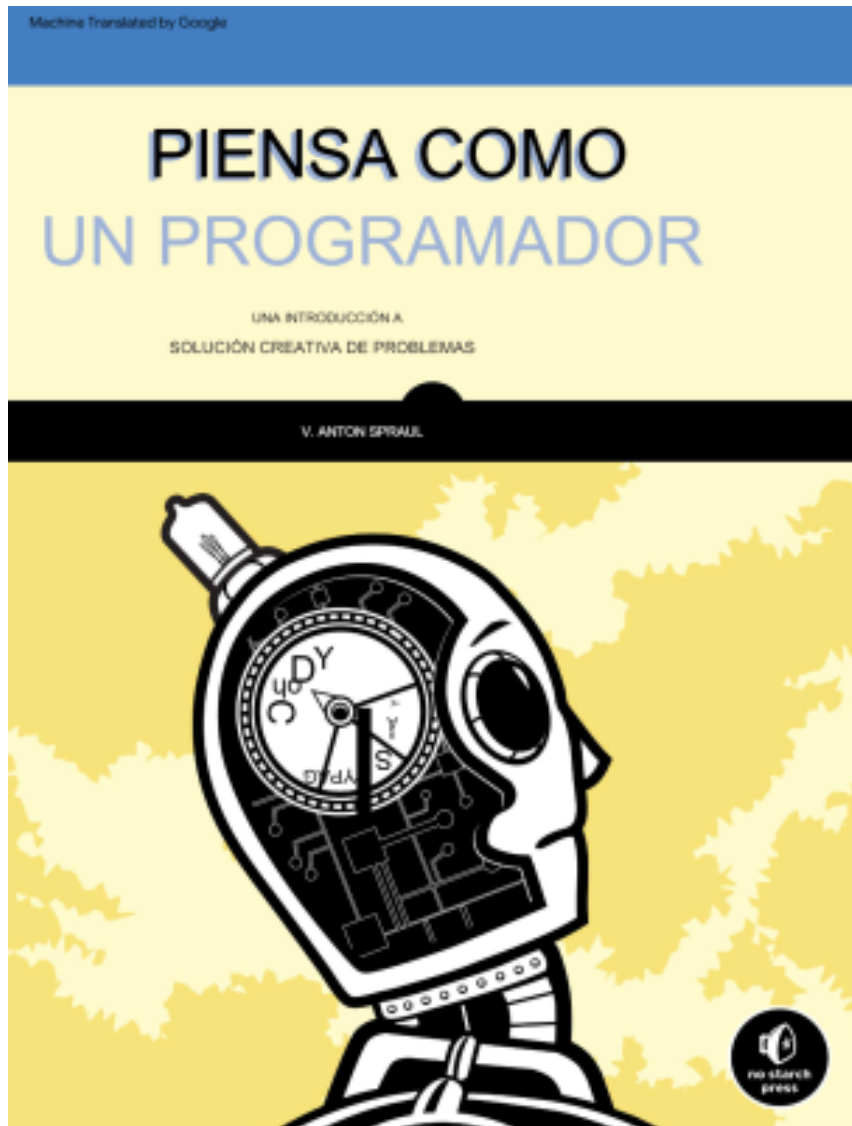
¿PODRÍAS REPLICAR ESA ESTRATEGIA EN OTRAS SITUACIONES PROBLEMÁTICAS ?

¿QUÉ OCURRIÓ CUANDO LO HICISTE CON UN PAR ?.

Desafío:

Ahora Intentamos resolverlos de una forma más estructurada.

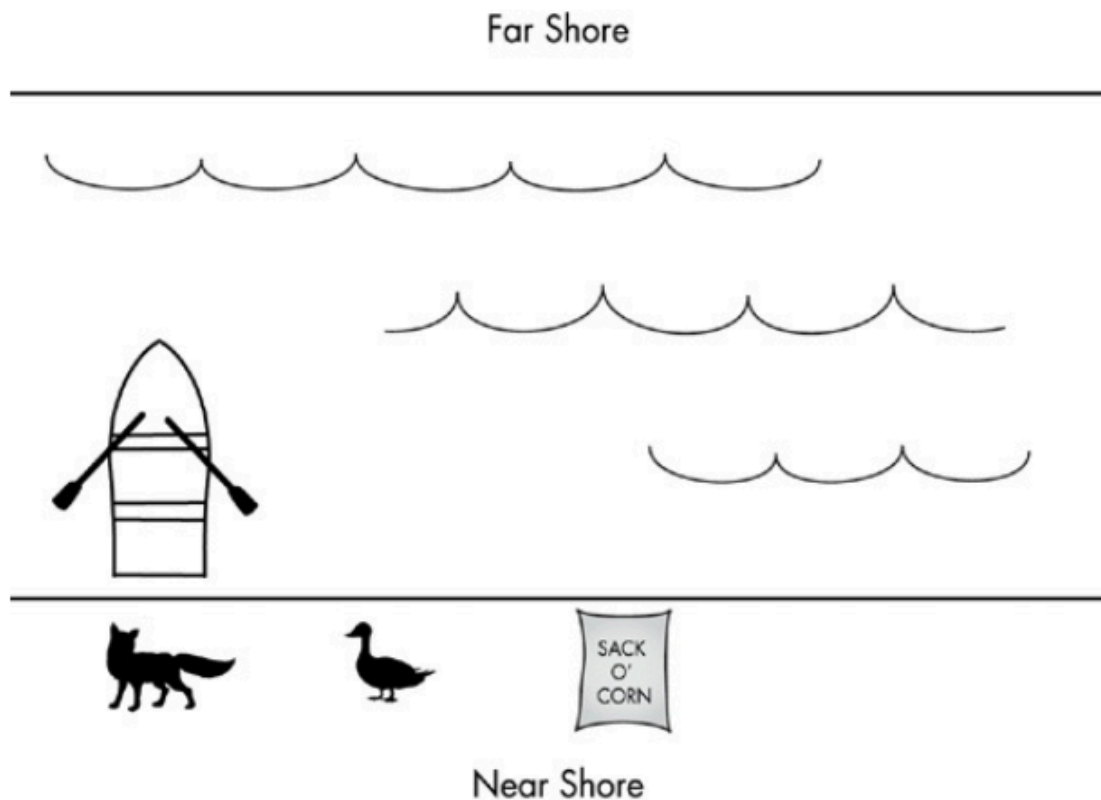
"Piensa como un programador: Una introducción a la resolución creativa de problemas"



<https://teachinglondoncomputing.org/puzzles/>

Los problemas incluyen **restricciones**, reglas inquebrantables sobre el problema o la forma en que debe resolverse el problema.

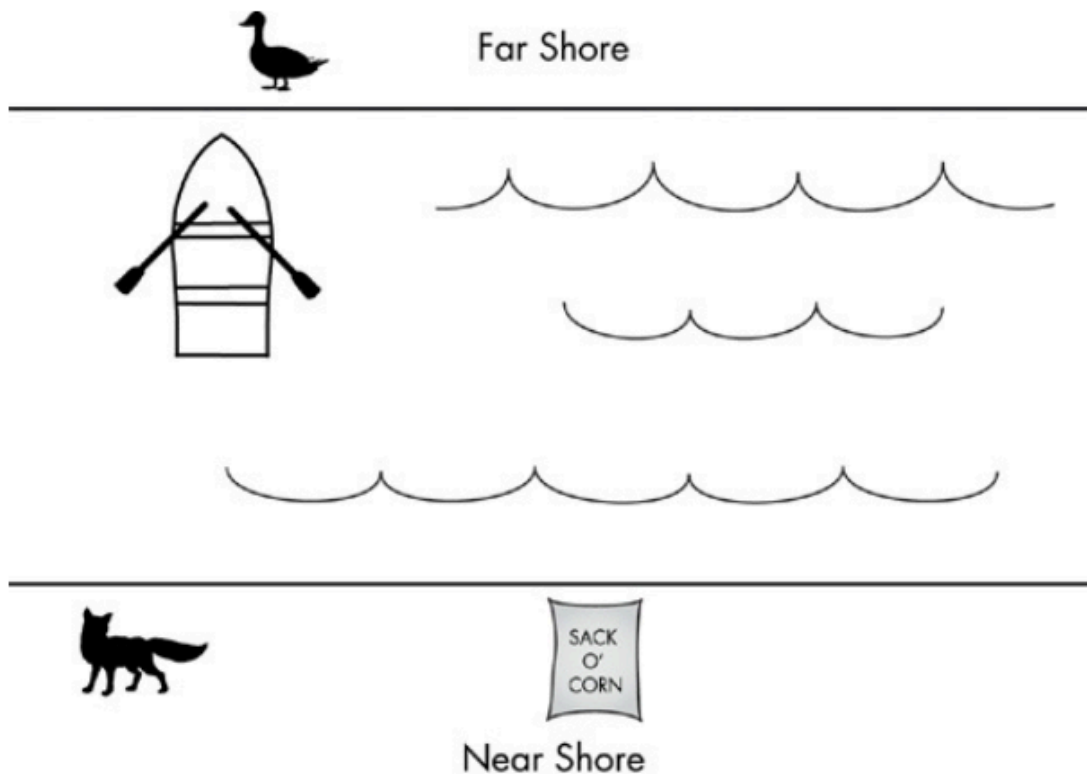
Entonces, para los programadores podemos definir la resolución de problemas como escribir un programa original que realice un conjunto particular de tareas y cumpla todas las restricciones establecidas.



Pocas personas son capaces de resolver este acertijo, al menos sin una pista. Yo sé que no lo era.

Así es como suele funcionar el razonamiento. Como el granjero solo puede llevarse una cosa a la vez, necesitará varios viajes para llevarlo todo a la otra orilla. En el primer viaje, si el granjero se lleva al zorro, el ganso se quedaría con el saco de maíz y el ganso se lo comería. Del mismo modo, si el granjero se lleva el saco de maíz en el primer viaje, el zorro se quedaría con el ganso y este se lo comería.

Por lo tanto, el granjero debe llevarse al ganso en el primer viaje, lo que resulta en la configuración que se muestra en la Figura 1-2.



Hasta aquí, todo bien. Pero en el segundo viaje, el granjero debe llevarse el zorro o el maíz.

Sin embargo, lo que el granjero se lleve debe dejarlo en la orilla opuesta con el ganso, mientras que el granjero regresa a la orilla cercana por el objeto restante. Esto significa que o bien el zorro y el ganso se quedarán juntos, o bien el ganso y el maíz se quedarán juntos. Como ninguna de estas situaciones es aceptable, el problema parece irresoluble.

De nuevo, si ya has visto este problema, probablemente recuerdes el elemento clave de la solución. El granjero debe llevarse el ganso en el primer viaje, como se explicó anteriormente. En el segundo viaje, supongamos que el granjero se lleva al zorro. En lugar de dejar al zorro con el ganso, el granjero lleva el ganso de vuelta a la orilla cercana. Luego, el granjero cruza el saco de maíz, dejando al zorro y el maíz en la orilla opuesta, mientras regresa para un cuarto viaje con el ganso. La solución completa se muestra en la Figura 1-3.

Este acertijo es difícil porque la mayoría de la gente nunca considera llevar uno de los objetos de la orilla opuesta a la cercana. Algunos incluso sugieren que el problema es injusto, diciendo algo como: "¡No dijiste que podía llevarme algo!". Esto es cierto, pero también es cierto que nada en la descripción del problema sugiere que llevar algo esté prohibido.

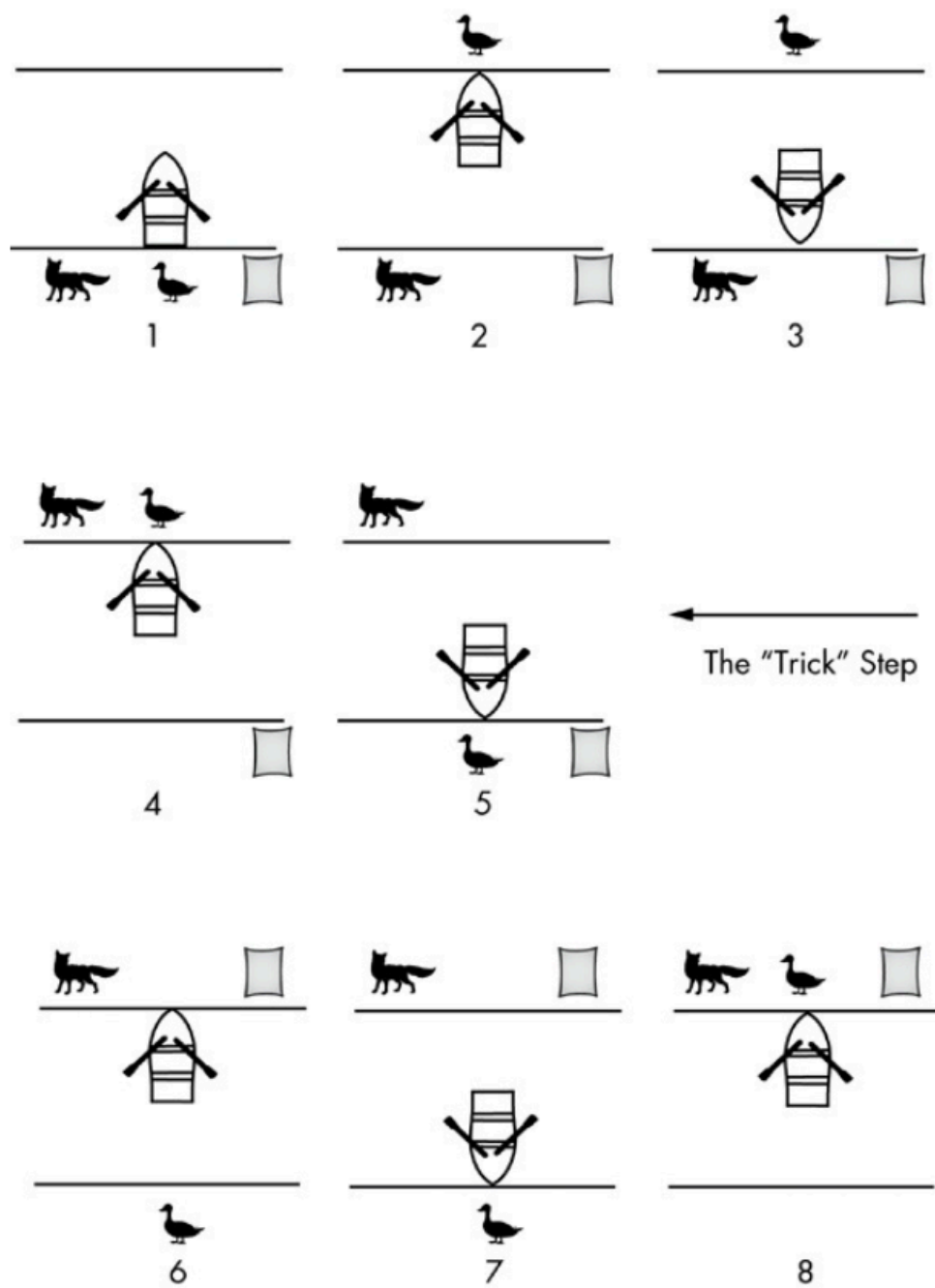


Figure 1-3. Step-by-step solution to the fox, goose, and corn puzzle

Piensa en cuánto más fácil sería resolver el rompecabezas si se hiciera explícita la posibilidad de llevar uno de los objetos a la orilla cercana: el granjero tiene un bote de remos que puede usar para transferir objetos en cualquier dirección, pero solo hay espacio para el granjero y uno de sus tres objetos. Con esa sugerencia a la vista, más personas resolverían el problema. **Esto ilustra un principio importante de la resolución de problemas: si desconoces todas las posibles acciones que podrías realizar, es posible que no puedas resolver el problema.**

Podemos referirnos a estas acciones como operaciones.

Al enumerar todas las posibles operaciones, podemos resolver muchos problemas probando cada combinación de operaciones hasta encontrar una que funcione. De manera más general, al reformular un problema en términos más formales, a menudo podemos descubrir soluciones que de otro modo se nos habrían escapado.

Olvidemos que ya conocemos la solución e intentemos plantear este rompecabezas en particular de forma más formal.

Primero, enumeraremos nuestras restricciones. Las restricciones clave son:

1. El granjero solo puede llevar un objeto a la vez en el bote.
2. El zorro y el ganso no pueden quedarse solos en la misma orilla.
3. El ganso y el maíz no pueden quedarse solos en la misma orilla.

Este problema es un buen ejemplo de la importancia de las restricciones. Si eliminamos cualquiera de estas restricciones, el rompecabezas se simplifica. Si eliminamos la primera restricción, podemos simplemente llevar los tres objetos al otro lado en un solo viaje. Incluso si solo podemos llevar dos objetos en el bote, podemos llevar el zorro y el maíz al otro lado y luego regresar por el ganso. Si eliminamos la segunda restricción (pero dejamos las demás), solo debemos tener cuidado: primero llevamos el ganso al otro lado, luego el zorro y finalmente el maíz.

Por lo tanto, si olvidamos o ignoramos alguna de las restricciones, terminaremos con un Kobayashi Maru.

A continuación, enumeramos las operaciones. Hay varias maneras de formular las operaciones para este rompecabezas. Podríamos hacer una lista específica de las acciones que creemos que podemos realizar:

1. Operación: Llevar al zorro al otro lado del río.
2. Operación: Llevar al ganso al otro lado del río.
3. Operación: Llevar el maíz al otro lado del río.

Recuerde, sin embargo, que el objetivo de reformular formalmente el problema es obtener información para una solución. A menos que ya hayamos resuelto el problema y descubierto la posible operación "oculta", llevar al ganso de vuelta al otro lado del río, no la descubriremos al hacer nuestra lista de acciones. En su lugar, deberíamos intentar que las operaciones sean genéricas o parametrizadas.

1. Operación: Remar el bote de una orilla a la otra.
2. Operación: Si el bote está vacío, cargar un objeto de la orilla.
3. Operación: Si el bote no está vacío, descargar el objeto en la orilla.

Al considerar el problema en términos generales, esta segunda lista de operaciones nos permitirá resolverlo sin necesidad de un momento de sorpresa con respecto al viaje de regreso a la orilla cercana con el ganso. Si generamos todas las posibles secuencias de movimientos, finalizando cada secuencia cuando viole una de nuestras restricciones o alcance una configuración que ya hayamos visto, finalmente daremos con la secuencia de la Figura 1-3 y resolveremos el rompecabezas. La dificultad inherente del rompecabezas se habrá superado mediante la reformulación formal de las restricciones y las operaciones.

Lecciones aprendidas

¿Qué podemos aprender del zorro, el ganso y el maíz?

Replantar el problema de forma más formal es una excelente técnica para comprenderlo mejor. Muchos programadores buscan a otros programadores para discutir un problema, no solo porque otros programadores puedan tener la respuesta, sino también porque expresarlo en voz alta suele generar ideas nuevas y útiles. Replantar un problema es como tener esa conversación con otro programador, solo que se interpretan ambos papeles.

La lección más importante es que pensar en el problema puede ser tan productivo, o en algunos casos más, que pensar en la solución. En muchos casos, el enfoque correcto para la solución es la solución.

Técnica del pato de goma

<https://leonardonarvaez.com/blog/detail/conoces-la-tecnica-del-pato-de-goma/>



Rubber Ducking

A very simple but particularly useful technique for finding the cause of a problem is simply to explain it to someone else. The other person should look over your shoulder at the screen, and nod his or her head constantly (like a rubber duck bobbing up and down in a bathtub). They do not need to say a word; the simple act of explaining, step by step, what the code is supposed to do often causes the problem to leap off the screen and announce itself.⁷

Sudoku

El sudoku se ha vuelto enormemente popular gracias a su aparición en periódicos y revistas, así como en la web y en el móvil.

Existen variantes, pero analizaremos brevemente la versión tradicional.

PROBLEMA: COMPLETAR UN CUADRADO DE SUDOKU

Una cuadrícula de 9×9 está parcialmente llena con dígitos individuales (del 1 al 9), y el jugador debe rellenar las casillas vacías cumpliendo ciertas condiciones: en cada fila y columna, cada dígito debe aparecer exactamente una vez, y además, en cada área marcada de 3×3, cada dígito debe aparecer exactamente una vez.

Si ya has jugado a este juego, probablemente ya tengas un conjunto de estrategias para completar un cuadrado en el menor tiempo posible.

Centrémonos en la estrategia inicial clave observando el cuadrado de ejemplo que se muestra en la Figura 1-11.

	9	1		6		7		
				8	2		3	9
5		3				2		
			9	1	3		6	2
		2	4		6	8		
1	4		8	2	5			
		9				5		7
6	7		1	5				
		5		4		6	9	

Figure 1-11. An easy sudoku square puzzle

Los sudokus varían en dificultad, determinada por el número de casillas que queden por completar. Por esta razón, este es un sudoku muy fácil. Como 36 casillas ya están numeradas, solo quedan 45 por completar. La pregunta es: ¿qué casillas debemos intentar completar primero? Recuerda las restricciones del sudoku. Cada uno de los nueve dígitos debe aparecer una vez en cada fila, en cada columna y en cada área de 3×3 marcada con bordes gruesos.

Estas reglas indican dónde debemos empezar. El área de 3×3 del centro del sudoku ya tiene números en ocho de sus nueve casillas. Por lo tanto, la casilla del centro solo puede tener un valor posible: el que no esté representado en otra casilla de esa área. Ahí es donde debemos empezar a resolver este sudoku. El número que falta en esa área es el 7, así que lo colocaríamos en la casilla del medio.

Con ese valor establecido, observe que la columna central ahora tiene valores en siete de sus nueve casillas, lo que deja solo dos casillas restantes, cada una de las cuales debe tener un valor que no esté ya en la columna: Los dos números que faltan son 3 y 9. La restricción en esta columna nos permitiría colocar cualquier número en cualquier lugar, pero observe que el 3 ya está presente en la tercera fila y el 9 en la séptima. Por lo tanto, las restricciones de fila dictan que el 9 debe ir en la tercera fila de la columna central y el 3 en la séptima. Estos pasos se resumen en la Figura 1-12.

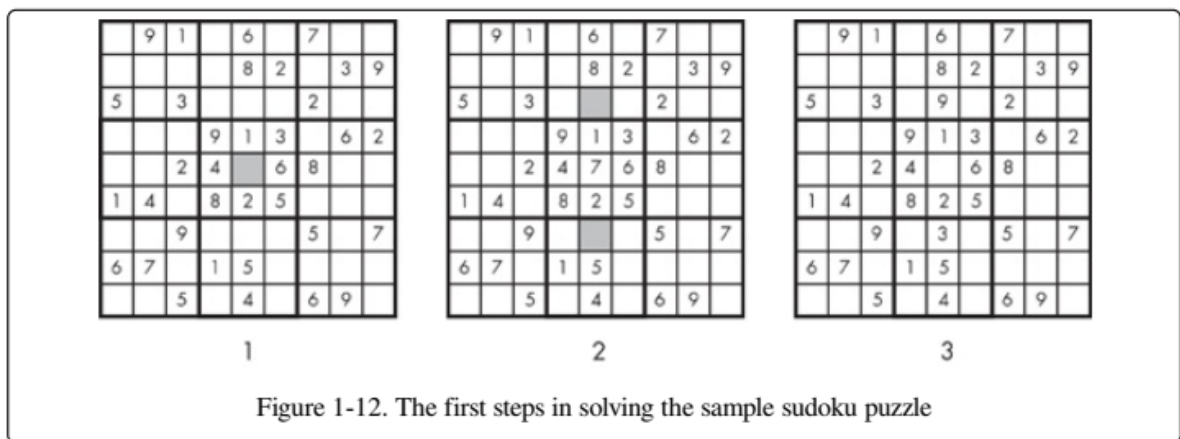


Figure 1-12. The first steps in solving the sample sudoku puzzle

No resolveremos todo el rompecabezas aquí, pero estos primeros pasos nos indican que debemos buscar cuadrados que tengan el menor número de valores posibles; idealmente, solo uno.

Lecciones aprendidas

La principal lección del sudoku es que debemos buscar la parte más restringida del problema. Si bien las restricciones suelen dificultar un problema al principio (recuerde el zorro, el ganso y el maíz), también pueden simplificar nuestra comprensión de la solución porque eliminan opciones.

Aunque no abordaremos la inteligencia artificial específicamente en este libro, existe una regla para resolver ciertos tipos de problemas de inteligencia artificial llamada "variable más restringida". Esto significa que, en un problema en el que se intenta asignar diferentes valores a distintas variables para cumplir con las restricciones, se debe comenzar con la variable con más restricciones, o dicho de otro modo, la variable con el menor número de valores posibles.

He aquí un ejemplo de este tipo de razonamiento. Supongamos que un grupo de compañeros de trabajo quiere ir a almorzar juntos y te piden que encuentres un restaurante que les guste a todos. El problema es que cada uno impone algún tipo de restricción a la decisión grupal: Pam es vegetariana, a Todd no le gusta la comida china, etc. Si tu objetivo es minimizar el tiempo que lleva encontrar un restaurante, deberías empezar por hablar con el compañero con las restricciones más exigentes. Si Bob tiene varias alergias alimentarias, por ejemplo, sería lógico empezar por encontrar una lista de restaurantes donde sabe que puede comer, en lugar de empezar con Todd, cuya aversión a la comida china se puede mitigar fácilmente. Esta misma técnica se puede aplicar a menudo a problemas de programación. Si una parte del problema tiene muchas restricciones, es un buen punto de partida, ya que puedes avanzar sin preocuparte por perder tiempo en trabajo que luego se deshará. Un corolario relacionado es que debes empezar por lo obvio. Si puedes resolver parte del problema, sigue adelante y haz lo que puedas. Puedes aprender algo al ver tu propio código que estimulará tu imaginación para resolver el resto.

Técnicas generales de resolución de problemas

Los ejemplos que hemos analizado demuestran muchas de las técnicas clave que se emplean en la resolución de problemas. En el resto de este libro, analizaremos problemas específicos de programación y encontraremos maneras de resolverlos, pero primero necesitamos un conjunto general de técnicas y principios. Algunas áreas problemáticas tienen técnicas específicas, como veremos, pero las reglas que se presentan a continuación se aplican a casi cualquier situación. **Si las incorporas regularmente a tu enfoque de resolución de problemas, siempre tendrás un método para abordar un problema.**

Ten siempre un plan

Esta es quizás la regla más importante. Siempre debes tener un plan, en lugar de embarcarte en actividades sin rumbo.

Llegados a este punto, deberías comprender que tener un plan siempre es posible. Es cierto que si no has resuelto el problema mentalmente, no puedes tener un plan para implementar una solución en código. Eso vendrá después. Sin embargo, incluso al principio, debes tener un plan para encontrar la solución.

Para ser justos, el plan puede requerir modificaciones en algún momento del proceso, o puede que tengas que abandonar el plan original y crear otro. ¿Por qué, entonces, es tan importante esta regla? El general Dwight D. Eisenhower fue famoso por decir: «Siempre he creído que los planes son inútiles, pero la planificación es indispensable». Quería decir que las batallas son tan caóticas que es imposible predecir todo lo que podría suceder y tener una respuesta predeterminada para cada resultado. En ese sentido, los planes son inútiles en el campo de batalla (otro líder militar, el prusiano Helmuth von Moltke, dijo la famosa frase de que «ningún plan sobrevive al primer contacto con el enemigo»). Pero ningún ejército puede tener éxito sin planificación y organización. A través de la planificación, un general

aprende cuáles son las capacidades de su ejército, cómo trabajan juntas las diferentes partes del ejército, etc.

De la misma manera, siempre debes tener un plan para resolver un problema. Puede que no sobreviva al primer contacto con el enemigo —puede que lo descartes en cuanto empieces a escribir código en tu editor de código fuente—, pero debes tener un plan.

Sin un plan, simplemente estás esperando un golpe de suerte, el equivalente a que un mono escriba al azar y produzca una obra de Shakespeare. Los golpes de suerte son poco comunes, y los que ocurren pueden requerir un plan. Mucha gente ha oído la historia del descubrimiento de la penicilina: un investigador llamado Alexander Fleming olvidó cerrar una placa de Petri una noche y a la mañana siguiente descubrió que el moho había inhibido el crecimiento de las bacterias en la placa. Pero Fleming no se quedó esperando un golpe de suerte; había estado experimentando de forma exhaustiva y controlada, y por lo tanto reconoció la importancia de lo que vio en la placa de Petri. (Si encontrara moho creciendo en algo que dejó fuera la noche anterior, esto no supondría una contribución importante a la ciencia).

La planificación también te permite establecer objetivos intermedios y alcanzarlos. Sin un plan, solo tienes un objetivo: resolver el problema completo. Hasta que no lo resuelvas, no sentirás que has logrado nada. Como probablemente hayas experimentado, muchos programas no aportan nada útil hasta que están cerca de su finalización. Por lo tanto, trabajar solo por el objetivo principal inevitablemente conduce a la frustración, ya que no hay un refuerzo positivo de tus esfuerzos hasta el final. Si, en cambio, creas un plan con una serie de objetivos menores, incluso si algunos parecen tangenciales al problema principal, lograrás un progreso medible hacia una solución y sentirás que has invertido tu tiempo de forma útil. Al final de cada sesión de trabajo, podrás ir completando los elementos de tu plan, con la confianza de que encontrarás una solución en lugar de frustrarte cada vez más.

Replantear el problema

Como lo demuestra especialmente el problema del zorro, el ganso y el maíz, replantear un problema puede producir resultados valiosos. En algunos casos, un problema que parece muy difícil puede parecer fácil al plantearse de otra manera o con otros términos.

Replantear un problema es como rodear la base de una colina que debes escalar; antes de comenzar, ¿por qué no examinar la colina desde todos los ángulos para ver si hay una manera más fácil de subir?

La reformulación a veces nos muestra que el objetivo no era el que pensábamos. Una vez leí sobre una abuela que cuidaba a su nieta mientras tejía. Para terminar de tejer, la abuela puso a la bebé a su lado en un corral portátil, pero a la bebé no le gustaba estar allí y no paraba de llorar. La abuela probó todo tipo de juguetes para que el corral fuera más divertido para la bebé, hasta que se dio cuenta de que mantenerla dentro era solo un medio para un fin. El objetivo era que la abuela pudiera tejer tranquilamente. La solución: dejar que la bebé jugara feliz en la alfombra mientras la abuela tejía dentro del corral.

La reformulación puede ser una técnica poderosa, pero muchos programadores la omiten porque no implica directamente escribir código ni siquiera diseñar una solución. Esta es otra razón por la que tener un plan es esencial. Sin un plan, el único objetivo es tener un código funcional, y la reformulación le quita tiempo a la escritura de código. Con un plan, puedes indicar “replantear formalmente el problema” como tu primer paso; por lo tanto, completar la reformulación se considera oficialmente un avance.

Incluso si una reformulación no genera una comprensión inmediata, puede ser útil de otras maneras. Por ejemplo, si un supervisor o instructor le ha asignado un problema, puede presentar su reformulación a quien lo asignó y confirmar su comprensión. Además, reformular el problema puede ser un prerrequisito necesario para usar otras técnicas comunes, como reducirlo o dividirlo.

En términos más generales, la reformulación puede transformar áreas completas del problema. La técnica que utilizo para las soluciones recursivas, que comparto en un capítulo posterior, es un método para reformular problemas recursivos y así poder tratarlos de la misma manera que los problemas iterativos.

Dividir el problema

Encontrar una manera de dividir un problema en pasos o fases puede simplificarlo mucho. Si se puede dividir un problema en dos partes, se podría pensar que cada parte sería la mitad de difícil de resolver que el conjunto original, pero normalmente es incluso más fácil.

He aquí una analogía que resultará familiar si ya conoce algoritmos de ordenación comunes. Supongamos que tiene 100 archivos que necesita colocar en una caja en orden alfabético, y que su método básico de ordenación alfabética es, en efecto, lo que llamamos ordenación por inserción: se toma un archivo al azar, se coloca en la caja, luego se coloca el siguiente archivo en la caja en la relación correcta con el primer archivo, y luego se continúa, colocando siempre el nuevo archivo en su posición correcta con respecto a los demás, de modo que, en cualquier momento, los archivos de la caja estén ordenados alfabéticamente. Supongamos que alguien separa inicialmente los archivos en cuatro grupos de tamaño aproximadamente igual (A-F, G-M, N-S y T-Z) y le dice que ordene alfabéticamente los cuatro grupos individualmente y luego los coloque uno tras otro en la caja.

Si cada uno de los grupos contenía unos 25 archivos, se podría pensar que ordenar alfabéticamente 4 grupos de 25 es aproximadamente el mismo trabajo que ordenar alfabéticamente un solo grupo de 100. Pero en realidad es mucho menos trabajo, ya que el trabajo que implica insertar un solo archivo aumenta a medida que aumenta el número de archivos ya archivados; hay que mirar cada archivo en la caja para saber dónde debe colocarse el nuevo. (Si lo duda, piense en una versión más extrema: compare la idea de ordenar 50 grupos de 2 archivos, lo cual probablemente podría hacer en menos de un minuto, con ordenar un solo grupo de 100 archivos).

De la misma manera, dividir un problema a menudo puede reducir la dificultad en un orden de magnitud. Combinar técnicas de programación es mucho más complicado que usar técnicas solas. Por ejemplo, una sección de código que emplea una serie de sentencias if dentro de un bucle while que a su vez está dentro de un bucle for será más difícil de escribir (y de leer) que una sección de código que emplea todas esas mismas sentencias de control secuencialmente.

En los siguientes capítulos, analizaremos formas específicas de dividir problemas, pero siempre debe estar atento a esta posibilidad. Recuerde que algunos problemas, como nuestro rompecabezas de fichas deslizantes, suelen ocultar su posible subdivisión. A veces, la forma de encontrar las divisiones de un problema es reducirlo, como veremos en breve.

Empieza con lo que sabes.

A quienes escriben novelas por primera vez se les suele aconsejar "escribe sobre lo que sabes". Esto no significa que los novelistas deban intentar crear obras solo en torno a incidentes y personas que hayan observado directamente en sus vidas; si así fuera, nunca podríamos tener novelas de fantasía, ficción histórica ni muchos otros géneros populares. Pero sí significa que cuanto más se aleja un escritor de su propia experiencia, más difícil puede ser escribir.

De la misma manera, al programar, intenta empezar con lo que ya sabes hacer y trabajar a partir de ahí. Una vez que hayas dividido el problema en partes, por ejemplo, continúa y completa las que ya sabes codificar. Tener una solución parcial funcional puede inspirar ideas para el resto del problema. Además, como habrás notado, un tema común en la resolución de problemas es lograr avances útiles para generar confianza en que finalmente completarás la tarea. Al empezar con lo que sabes, generas confianza y te impulsas hacia el objetivo.

La máxima de "empieza con lo que sabes" también aplica cuando no has dividido el problema. Imagina que alguien hiciera una lista completa de todas las habilidades de programación: escribir una clase de C++, ordenar una lista de números, encontrar el valor máximo en una lista enlazada, etc. En cada etapa de tu desarrollo como programador, encontrarás en esta lista muchas habilidades que puedes dominar, otras que puedes usar con esfuerzo y otras que aún no dominas.

Un problema en particular puede ser totalmente solucionable con las habilidades que ya tienes o puede que no, pero deberías investigarlo a fondo usando las habilidades que ya tienes antes de buscar otra solución. Si consideramos las habilidades de programación como herramientas y un problema de programación como un proyecto de reparación en casa, deberías intentar hacer la reparación usando las herramientas que ya tienes en el garaje antes de ir a la ferretería. Esta técnica sigue los principios que ya hemos mencionado. Sigue un plan y ordena nuestros esfuerzos. Cuando comenzamos la investigación de un

problema aplicando las habilidades que ya tenemos, podemos aprender más sobre el problema y su solución final.

Reducir el problema

Con esta técnica, al enfrentarse a un problema que no puede resolver, reduce su alcance, ya sea añadiendo o eliminando restricciones, para generar un problema que sí sabe cómo resolver. Veremos esta técnica en acción en capítulos posteriores, pero aquí tiene un ejemplo básico. Suponga que recibe una serie de coordenadas en un espacio tridimensional y debe encontrar las coordenadas más cercanas entre sí. Si no sabe cómo resolverlo de inmediato, existen diferentes maneras de reducir el problema para buscar una solución. Por ejemplo, ¿qué sucede si las coordenadas están en un espacio bidimensional, en lugar de tridimensional? Si eso no funciona, ¿qué sucede si los puntos se encuentran a lo largo de una sola línea, de modo que las coordenadas son simplemente números individuales (por ejemplo, dobles de C++)? Ahora, la pregunta se convierte, en esencia, en una lista de números, en encontrar los dos números con la mínima diferencia absoluta.

O se podría reducir el problema manteniendo las coordenadas en un espacio tridimensional, pero con solo tres valores, en lugar de una serie de tamaño arbitrario. Así, en lugar de un algoritmo para hallar la distancia mínima entre dos coordenadas, se trata simplemente de comparar la coordenada A con la coordenada B, luego la B con la C, y finalmente la A con la C.

Estas reducciones simplifican el problema de diferentes maneras. La primera reducción elimina la necesidad de calcular la distancia entre puntos tridimensionales. Quizás aún no sepamos cómo hacerlo, pero hasta que lo averigüemos, podemos avanzar hacia una solución. La segunda reducción, en cambio, se centra casi por completo en calcular la distancia entre puntos tridimensionales, pero elimina el problema de encontrar un valor mínimo en una serie de valores de tamaño arbitrario.

Por supuesto, para resolver el problema original, eventualmente necesitaremos las habilidades involucradas en ambas reducciones. Aun así, la reducción nos permite trabajar en un problema más simple incluso cuando no podemos encontrar la manera de dividirlo en pasos. En efecto, es como un Kobayashi Maru deliberado, pero temporal. Sabemos que no estamos trabajando en el problema completo, pero el problema reducido tiene suficiente en común con el problema completo como para avanzar hacia la solución definitiva. Muchas veces, los programadores descubren que poseen todas las habilidades individuales necesarias para resolver el problema y, al escribir código para resolver cada aspecto del problema, ven cómo combinar las distintas piezas de código en un todo unificado.

Reducir el problema también nos permite identificar con precisión dónde reside la dificultad restante. Los programadores principiantes a menudo necesitan buscar la ayuda de programadores experimentados, pero esto puede ser una experiencia frustrante para todos los involucrados si el programador con dificultades no puede describir con precisión la ayuda que necesita. Uno nunca quiere verse obligado a decir: "Aquí está mi programa y no funciona. ¿Por qué no?". Usando la técnica de reducción de problemas, uno puede identificar con precisión la ayuda necesaria, diciendo algo como: "Aquí hay un código que escribí. Como pueden ver, sé cómo hallar la distancia entre dos coordenadas tridimensionales y sé cómo comprobar si una distancia es menor que otra. Pero no puedo encontrar una solución general para hallar el par de coordenadas con la distancia mínima".

Busque analogías

Una analogía, para nuestros propósitos, es una similitud entre un problema actual y un problema ya resuelto que puede aprovecharse para ayudar a resolver el problema actual. La similitud puede adoptar muchas formas. A veces significa que los dos problemas son en realidad el mismo. Esta es la situación que tuvimos con el problema del zorro, el ganso y el maíz y el problema de la cerradura de Quarrasi.

La mayoría de las analogías no son tan directas. A veces, la similitud solo afecta a una parte de los problemas. Por ejemplo, dos problemas de procesamiento de números pueden ser diferentes en todos los aspectos, excepto que ambos trabajan con números que requieren mayor precisión que la proporcionada por los tipos de datos de punto flotante integrados; no podrá usar esta analogía para resolver el problema completo, pero si ya ha encontrado una manera de abordar el problema de la precisión adicional, puede abordarlo de la misma manera.

Aunque reconocer analogías es la forma más importante de mejorar tu velocidad y habilidad para resolver problemas, también es la habilidad más difícil de desarrollar. La razón por la que al principio es tan difícil es que no puedes buscar analogías hasta tener un conjunto de soluciones previas como referencia. Aquí es donde los programadores en desarrollo a menudo intentan tomar un atajo, encontrando código similar al código necesario y modificándolo a partir de ahí. Sin embargo, por varias razones, esto es un error. En primer lugar, si no completas una solución tú mismo, no la habrás comprendido ni asimilado por completo. En pocas palabras, es muy difícil modificar correctamente un programa que no comprendes del todo. No necesitas haber escrito código para comprenderlo completamente, pero si no pudieras haberlo escrito, tu comprensión será necesariamente limitada. En segundo lugar, cada programa exitoso que escribes es más que una solución a un problema actual; es una fuente potencial de analogías para resolver problemas futuros. Cuanto más dependas del código de otros programadores ahora, más tendrás que depender de él en el futuro. Hablaremos en profundidad sobre la "reutilización adecuada" y la "reutilización inadecuada" en el capítulo 7.

Experimenta

A veces, la mejor manera de progresar es probar cosas y observar los resultados.

Tenga en cuenta que experimentar no es lo mismo que adivinar. Cuando adivina, escribe un código y espera que funcione, sin tener una convicción firme de que así será. Un experimento es un proceso controlado. Se formula una hipótesis sobre qué ocurrirá al ejecutar cierto código, se prueba y se comprueba si la hipótesis es correcta. A partir de estas observaciones, se obtiene información que ayudará a resolver el problema original.

La experimentación puede ser especialmente útil al trabajar con interfaces de programación de aplicaciones o bibliotecas de clases. Supongamos que está escribiendo un programa que utiliza una clase de biblioteca que representa un vector (en este contexto, un array unidimensional que crece automáticamente a medida que se añaden elementos), pero nunca ha usado esta clase de vector y no está seguro de qué sucede cuando se elimina un elemento del vector. En lugar de seguir adelante con la solución del problema original mientras las incertidumbres le rondan la cabeza, podría crear un programa corto e independiente solo para experimentar con la clase de vector y probar específicamente las situaciones que le preocupan. Si dedica algo de tiempo al programa "demostrador de vectores", podría convertirse en una referencia para futuros trabajos con la clase.

No te frustres

La última técnica no es tanto una técnica, sino una máxima: No te frustres.

Cuando te frustras, no piensas con claridad, no trabajas con la misma eficiencia y todo te llevará más tiempo y te parecerá más difícil. Peor aún, la frustración tiende a alimentarse, de modo que lo que empieza como una leve irritación termina en ira pura.

Cuando doy este consejo a los nuevos programadores, suelen replicar que, si bien en principio están de acuerdo con mi punto de vista, no tienen control sobre sus frustraciones. ¿No es pedirle a un programador que no se frustre por la falta de éxito como pedirle a un niño pequeño que no grite si pisa una tachuela? La respuesta es no. Cuando alguien pisa una tachuela, se envía inmediatamente una fuerte señal a través del sistema nervioso central, donde las capas más profundas del

cerebro responden. A menos que sepas que estás a punto de pisar la tachuela, es imposible reaccionar a tiempo para contrarrestar la respuesta automática del cerebro. Así que dejaremos que el pequeño se libre de gritar.

El programador no está en la misma situación. A riesgo de parecer un experto en autoayuda, un programador frustrado no responde a un estímulo externo. El programador frustrado no está enojado con el código fuente en el monitor, aunque pueda expresar su frustración en esos términos. En cambio, está enojado consigo mismo. La fuente de la frustración es también el destino: la mente del programador.

Cuando te permites frustrarte —y uso la palabra "permitir" deliberadamente—, en realidad te estás dando una excusa para seguir fracasando. Imagina que estás trabajando en un problema difícil y sientes que tu frustración aumenta. Horas después, recuerdas una tarde de dientes apretados y lápices rotos por la ira y te dices a ti mismo que habrías avanzado mucho si hubieras podido calmarte. En realidad, puede que hayas decidido que ceder ante tu ira era más fácil que afrontar el problema.

En definitiva, evitar la frustración es una decisión que debes tomar. Sin embargo, hay algunas ideas que puedes aplicar y que te ayudarán. En primer lugar, nunca olvides la primera regla: siempre debes tener un plan, y si bien escribir código que resuelva el problema original es el objetivo de ese plan, no es el único paso. Por lo tanto, si tienes un plan y lo sigues, estás progresando y debes creerlo. Si has seguido todos los pasos de tu plan original y aún no estás listo para empezar a programar, es hora de crear otro.

Además, cuando llegue el momento de frustrarte o tomarte un descanso, deberías tomarte un descanso. Un truco es tener más de un problema en el que trabajar, así si este te bloquea, puedes centrar tus esfuerzos en otra cosa. Ten en cuenta que si divides el problema con éxito, puedes usar esta técnica en un solo problema; simplemente bloquea la parte del problema que te bloquea y trabaja en otra cosa. Si

no tienes otro problema que puedas abordar, levántate de la silla y haz otra cosa, algo que te mantenga la circulación pero que no te duela la cabeza: da un paseo, lava la ropa, haz tu rutina de estiramientos (si te estás apuntando a programador y te pasas el día sentado frente al ordenador, ¡te recomiendo encarecidamente que desarrolles una rutina de estiramientos!). No pienses en el problema hasta que termine tu descanso.

Practica Practica Practica ...

Recuerda, para aprender algo de verdad hay que ponerlo en práctica, así que realiza tantos ejercicios como puedas. En este primer capítulo, por supuesto, todavía no hablamos de programación, pero aun así, te animo a que pruebes algunos ejercicios. Piensa en estas preguntas como un calentamiento para tus dedos antes de empezar a tocar la música de verdad.

1. Intenta un sudoku de dificultad media (puedes encontrarlos en internet y probablemente en tu periódico local), experimenta con diferentes estrategias y anota los resultados. ¿Puedes escribir un plan general para resolver un sudoku?
2. Considera una variante del rompecabezas de fichas deslizantes donde las fichas estén cubiertas con una imagen en lugar de números. ¿Cuánto aumenta esto la dificultad y por qué?
3. Encuentra una estrategia para los rompecabezas de fichas deslizantes diferente a la mía.
4. Busca rompecabezas clásicos del tipo zorro, ganso y maíz e intenta resolverlos. Muchos de los grandes rompecabezas fueron creados o popularizados por Sam Loyd, así que podrías buscar su nombre. Además, una vez que descubras (o te rindas y leas) la solución, piensa en cómo podrías hacer una versión más sencilla del rompecabezas. ¿Qué tendrías que cambiar? ¿Las restricciones o simplemente la redacción?

5. Intenta escribir estrategias explícitas para otros juegos tradicionales de lápiz y papel, como los crucigramas. ¿Por dónde empezar? ¿Qué hacer cuando te quedes atascado? Incluso los juegos de periódico sencillos, como "Jumble" (palabras cruzadas, etc), son útiles para reflexionar sobre la estrategia.